Steam Simulator: Installation & Test Plan

The installation ships in two phases.

- **Phase 1 (now):** Python server + I/O App + Profibus hardware. You can

  install and verify these today using a browser to manually drive outputs and read inputs. No Pi, no simulator.

- **Phase 2 (later):** Raspberry Pi feed + simulator. Polina and Richard

  will ship updated `server.py`, the simulator EXE, and a refreshed `server.cfg.json` once the Pi-side software is updated to match the XML configuration in use at your site. Phase 1 components stay in place; Phase 2 layers on top.

This document covers Phase 1 in full. Phase 2 instructions are stubbed in at the end and will be filled in when that software ships.

PHASE 1: Python server + I/O App + Profibus

What you're installing

1. **Python server** - middleman that holds I/O state in memory and

   exposes browser pages for manual input/output control.

2. **I/O App** (`IOApp.exe`) - bridges the Python server and the

   Profibus cards. Reads Profibus inputs and posts them to the server; reads outputs from the server and writes them to Profibus.

The simulator and the Raspberry Pi are **not** part of Phase 1.

Step 1: Install Python on XP

Windows XP can only run **Python 3.4** (the last version that supports XP).

1. On a computer with internet, download:

`https://www.python.org/ftp/python/3.4.4/python-3.4.4.msi`

2. Copy the installer to a USB stick.
3. Plug the USB stick into the XP machine.
4. Run the installer. **Important:** at the "Customize" screen, scroll to the bottom and enable **"Add python.exe to Path"**.
5. Click through to finish.

Verify: in `cmd`, run `python --version`. You should see `Python 3.4.4`.

Step 2: Copy the Phase 1 files to XP

Copy these onto your USB stick:

```
IOApp.exe                                   (the I/O bridge)
IOApp.cfg                                   (Profibus block configuration)
io_server\
    server.py                               (the Python middleman)
    server.cfg.json                         (upstream device config)
```

On the XP machine, place them inside `C:\Steam_Sim\` so the final paths are:

```
C:\Steam_Sim\IOApp.exe
C:\Steam_Sim\IOApp.cfg
C:\Steam_Sim\io_server\server.py
C:\Steam_Sim\io_server\server.cfg.json
```

The simulator's existing runtime data (`C:\Steam_Sim\Text\`, `C:\Steam_Sim\Snapshots\`, etc.) stays where it is; it'll be used in Phase 2.

Required: empty the upstream devices list

Open `C:\Steam_Sim\io_server\server.cfg.json` in Notepad and replace the `devices` array with `[]`:

```
{
  "devices": []
}
```

This stops the server from trying to poll the Pi during Phase 1. (The default config points at a dev-side Pi that isn't reachable from your network, and even if it were, it speaks a JSON format that the

current server expects rather than the XML your Pi emits. Phase 2 ships an updated server that handles your Pi's format, along with the correct device entry.)

Optional: review IOApp.cfg

`C:\Steam_Sim\IOApp.cfg` is the Profibus block skip list. It defaults to skipping all analog input blocks (`cardA_ai_skip=all` , `cardB_ai_skip=all`) because the analog input Profibus hardware at CMA is currently broken. If a specific Profibus block needs to be excluded, add its address. Reread on every Start click in the I/O App.

Step 3: Run the Phase 1 system

Two windows.

Window 1 - Python server

1. **Start -> Run -> `cmd`** .
2. Type: `cd C:\Steam_Sim\io_server`
3. Type: `python server.py`
4. Leave this window open and visible. You should see output like:

   ``` Starting Python HTTP server on http://127.0.0.1:8080 ... Loaded 0 upstream device(s) from C:\Steam_Sim\io_server\server.cfg.json ``` "0 devices" is correct for Phase 1 - we cleared the list in Step 2.

Window 2 - I/O App

Double-click `C:\Steam_Sim\IOApp.exe` . The form opens with **Status: Stopped**.

Click **Start**. With Profibus cards present, you should see:

```
Card A initialized OK
Card B initialized OK
Starting I/O loop
```

If Profibus cards are not present (e.g., on a development laptop), you'll get pop-up errors saying "IO Card A failed to initialise" and the same for Card B. Dismiss both - this is expected without hardware. Status will show "Running (no cards)" and the rest of the system continues to work, but Tests 1 and 3 below will not pass (they require real Profibus hardware).

```

Step 4: Verify Phase 1

Three tests, in order. They exercise the full Profibus link end-to-end: hardware -> I/O App -> Python server -> browser, and back the other way.

Test 1: I/O App is reading Profibus inputs

Watch the I/O App log (Window 2). Every couple of seconds you should see a `tick` line:

```
12:34:56  tick  ain[0]=N aout[0]=N  din[0]=N dout[0]=N
```

The `ain` and `din` values should reflect whatever the Profibus cards are currently reading from physical sensors and switches.

Wiggle a physical input (toggle a switch, move a sensor). Within a couple of seconds the corresponding `ain[N]` or `din[N]` value in the log should change.

If values are stuck at zero: confirm the I/O App status is "Running" (not "Running (no cards)") and that the log isn't repeating "Card A read failed" or similar.

Test 2: Python server is receiving the Profibus inputs

In Firefox on the XP machine, navigate to:

```
http://127.0.0.1:8080/test/analog
```

The left column should show non-zero values for the Profibus-connected channels. Move a physical analog input on the simulator console - the corresponding row should update within ~1 second.

Then navigate to:

```
http://127.0.0.1:8080/test/digital
```

Toggle a physical switch - the corresponding row should flip from 0 to 1 (or back) within a second.

If the values stay at zero: check Window 1 for `POST /ioio/inputs` or `POST /ioio/outputs` lines from the I/O App. If you don't see any, the I/O App isn't reaching the server.

Test 3: Profibus outputs respond to server-driven commands

Skip if Profibus cards aren't connected.

In Firefox, navigate to:

```
http://127.0.0.1:8080/test/digital
```

Find a `dout` row that maps to a known lamp or relay. Set its value to 1 and submit. Within ~1 second:

- The I/O App log should show a `tick` line with the corresponding `dout[N]=1`.
- The physical lamp/relay should turn on.

Set it back to 0 and confirm it turns off.

Repeat with an `aout` row on `http://127.0.0.1:8080/test/analog` to exercise an analog output (gauge, motor, etc.). Try a few different values - the physical device should move proportionally.

If physical hardware doesn't respond: confirm the I/O App status shows "Running" (not "Running (no cards)") and that the log isn't repeating "Card A AO write failed" or similar.

Step 5: Shutting Phase 1 down

1. In the I/O App, click **Stop**, then close the window.
2. In the Python server console (Window 1), press **Ctrl+C**, then close the window.

Phase 1 troubleshooting

If a verification test doesn't pass, please collect the following and send to support:

1. **Python server console output.** Click in Window 1, then **right-click -> Mark**, drag-select all text, press **Enter** to copy, then paste into an email.
2. **I/O App log screenshot.** Take a screenshot of Window 2 with the log visible.
3. **Server response.** In Firefox on XP, hit `http://127.0.0.1:8080/ioio/status` and copy the response into an email.
4. **Network diagnostics.** In `cmd` on XP, run `ipconfig` and copy

the output.

PHASE 2: Raspberry Pi feed + simulator (coming later)

Phase 2 adds two more components on top of Phase 1:

- **Raspberry Pi feed** - your Pi's XML-formatted status response gets

  parsed by an updated Python server and used to drive analog/digital inputs that the Profibus side doesn't cover.

- **Simulator** (`Steam_SimV32_00_00_Disabled_Profibus.exe`) - the steam

  simulator UI and model, with Profibus reads/writes disabled internally so they go through the Python server instead.

When Phase 2 ships you'll receive:

- An updated `server.py` that reads XML from your Pi.
- A refreshed `server.cfg.json` with your Pi's address pre-filled.
- The simulator EXE.
- A revision of this document with installation steps for the new

  files, a new launch order (Python server -> I/O App -> Simulator), and end-to-end verification tests (Pi -> server -> simulator -> server -> I/O App -> Profibus).

The Phase 1 components stay where they are; Phase 2 layers on top without re-installing anything.